Contents

Fused causal attention with NVIDIA cuTile **1**
1. What the kernel does 1
2. How the kernel runs 1
3. Code map 2
4. Current evidence 2
5. Testing 3
6. Timing and profiling 3
7. Remaining work 4
8. Quick viva notes 4

Fused causal attention with NVIDIA cuTile

Project: IIT Madras GPU programming course
Target: Windows RTX 4060, compute capability 8.9
Main code: src/cutile_attention.py

1. What the kernel does

Causal attention computes

[O=\operatorname{softmax}\left(\frac{QK^T}{\sqrt D}+M\right)V]

where M masks keys after the current query position. Query i can use keys 0 through i, but not later tokens.

A standard implementation writes the full N-by-N score and probability matrices to GPU memory. This kernel works on query and key/value tiles instead. It keeps a running maximum, softmax sum and weighted-value accumulator for each query row, then writes the finished output tile.

Q/K/V and output use FP16. Score and softmax accumulation use FP32. The supported head dimensions are 32, 64 and 128. The current core path requires contiguous inputs, Dv = D, equal self-attention sequence lengths and sequence lengths divisible by both selected tile dimensions.

2. How the kernel runs

1. Validate the inputs and tile configuration.

2. Load a query tile and initialise the running softmax state.
3. Load key/value tiles in order.
4. Compute QK scores, scale them by 1/sqrt(D), and mask future keys.
5. Update the running maximum, normalisation sum and weighted-value accumulator.
6. Divide by the final sum and write the output.

For a new score tile with maximum mt, update the running maximum to mnew = max(m, mt). Rescale the old sum and accumulator by exp(m - mnew), then add the current tile using exp(scores - mnew). This keeps the softmax stable without storing all scores.

3. Code map

File	Purpose
src/cutile_attention.py	Main cuTile kernel and launch path
src/reference.py	Independent FP32 NumPy attention calcula
src/online_reference.py	Tiled online-softmax reference
src/contract.py	Input and launch checks
src/baseline.py	Deliberately unfused FP32 CuPy baseline
run_attention.py	Command-line interface
tests/test_gpu_correctness.py	Workload and tile comparisons
tests/test_gpu_edge_cases.py	Causal properties, numerical cases and inva
benchmarks/	Timing and profiler scripts

The ordinary CUDA C++ implementations are comparison controls, not cuTile implementations.

4. Current evidence

The last complete native RTX 4060 run was #39422402438, on commit bf5a623. It passed the 48 core GPU tests, three selected extension tests, 29 numerical configurations, CLI verification and three timing sessions. A separate NCU run saved eight reports and seven real GUI screenshots: #39414502490.

The kernel was compacted after that run. Rerun current main on the RTX 4060 before treating those results as validation of the current source. Existing numbers are historical until that succeeds.

The timing baseline is the deliberately unfused FP32 CuPy implementation. No speedup over FlashAttention or optimised PyTorch SDPA has been established.

5. Testing

Use the project virtual environment on the RTX 4060.

```
$py = ".\.venv-cutile-windows\Scripts\python.exe"

& $py scripts\cutile_smoke.py
if ($LASTEXITCODE -ne 0) { throw "cuTile runtime check failed."

$env:REQUIRE_CUTILE = "1"
& $py -m pytest -v -ra tests\test_gpu_correctness.py tests\test
$testCode = $LASTEXITCODE
Remove-Item Env:REQUIRE_CUTILE -ErrorAction SilentlyContinue
if ($testCode -ne 0) { throw "GPU tests failed." }
```

Create and check a CLI output:

```
& $py scripts\generate_inputs.py --output .tmp\sample.npz --bat
& $py run_attention.py --input .tmp\sample.npz --output .tmp\ou
& $py scripts\verify_cli_output.py --input .tmp\sample.npz --ou
```

The runtime check confirms cuTile can compile and launch on this machine. The 48 tests check attention behaviour. The separate 29-row matrix compares supported workload/tile combinations.

6. Timing and profiling

The full timing run uses seven workloads, five tile shapes where legal, 10 warmups, 30 CUDA-event samples and five launches per sample. Three sessions use rotated tile order. Report medians and variation together; don't pick a winner from one close result.

The saved NCU profiles showed high register and shared-memory use and low occupancy for the two sampled configurations. Scheduler data showed few eligible warps in many cycles. These are clues for further work, not proof of a single bottleneck. Check the memory, compute, scheduler and warp-state views together.

7. Remaining work

- Revalidate current main on the RTX 4060.
- Measure clean-cache first-call/JIT time for long specialisations.
- Complete a compatible framework attention comparison and record the backend.
- Save detailed NCU views and compare matched tile choices.
- Make one profile-driven change, then repeat correctness, timing and profiling.
- Update the report and record actual team contributions.

The course checklist, test matrix and evidence index track this work.

8. Quick viva notes

Why use tiles? They let the kernel reuse data and process the operation without writing the full score/probability matrices to global memory.

What makes the attention causal? Scores for future keys are masked before softmax.

Why online softmax? The kernel can combine tiles while keeping a stable running maximum, normalisation sum and weighted-value accumulator.

What is the current baseline? Unfused FP32 CuPy. It is not a vendor-optimised attention implementation.

What do the NCU results say? Resource use and scheduler data suggest limited occupancy and latency hiding. More than one counter is needed to identify the bottleneck.

Are all performance requirements closed? No. Current-source GPU validation, clean-cache JIT timing, the framework comparison and another measured profiler-guided improvement remain open.

See Report.md for the full measurement tables, failed attempts and detailed history.